

# TP Fil Rouge – Domain-Driven Design (DDD)

|         |                   |
|---------|-------------------|
| 👤 Owner | 👤 louis reynouard |
| 🏷️ Tags |                   |

## 0. Organisation générale

### 0.1. Modalités de travail

### 0.2. Repository Git

### 0.3. Règles générales

## Partie 1 : Découverte du domaine

### TP 1.1 — Choix du scénario et description du domaine

#### Objectif

#### Instructions

Livable attendu : [docs/domaine.md](#)

### TP 1.2 — Event Storming express

#### Objectif

#### Instructions

Livable attendu : [docs/event-storming.png](#)

### TP 1.3 — Glossaire (Ubiquitous Language v1)

#### Objectif

#### Instructions

Livable attendu : [docs/ubiquitous-language.md](#)

### TP 1.4 — Identification des sous-domaines

#### Objectif

#### Instructions

#### Livrables attendus

## Partie 2 : Bounded Contexts & Context Map

### TP 2.1 — Identification des Bounded Contexts

#### Objectif

#### Instructions

Livable attendu : [docs/bounded-contexts.md](#)

### TP 2.2 — Choix des patterns de relation & Context Map

#### Objectif

#### Instructions

Livable attendu : [docs/context-map.md](#)

### TP 2.3 — Ubiquitous Language v2

#### Objectif

#### Instructions

Livrable attendu : mise à jour de [docs/ubiquitous-language.md](#)

#### TP 2.4 — Alignement conceptuel du modèle

Objectif

Instructions

Livrable attendu : [docs/domain-concepts.md](#)

### Partie 3 : Modélisation tactique

#### TP 3.1 — Entités & Objets Valeur (modèle)

Objectif

Instructions

Livrable attendu : [docs/domain-model.md](#)

#### TP 3.2 — Agrégats & Invariants

Objectif

Instructions

Livrable attendu : [docs/invariants.md](#)

#### TP 3.3 — Tests de domaine (Given/When/Then textuels)

Objectif

Instructions

Livrable attendu : [docs/tests-domaine.md](#)

### Partie 4 : Architecture hexagonale (conceptuelle)

#### TP 4.1 — Squelette Hexagonal

Objectif

Instructions

Livrable attendu : [docs/architecture-hexagonale.md](#)

#### TP 4.2 — Repositories (conceptuels)

Objectif

Instructions

Livrable attendu : [docs/repositories.md](#)

#### TP 4.3 — Exposition REST (maquettes uniquement)

Objectif

Instructions

Livrable attendu : [docs/api-examples.md](#)

#### TP 4.4 — Tests d'intégration (scénarios sur papier)

Objectif

Instructions

Livrable attendu : [docs/integration-scenarios.md](#)

### Partie 5 : Intégration entre contextes & qualité

#### TP 5.1 — Définition des contrats

Objectif

Instructions

Livrable attendu : [docs/contracts.md](#)

#### TP 5.2 — Implémentation de l'intégration (design)

Objectif

Instructions

Livable attendu : <docs/integration-design.md>

#### TP 5.3 — Observabilité (conceptuelle)

Objectif

Instructions

Livable attendu : <docs/observability.md>

#### TP 5.4 — Validation par scénario

Objectif

Instructions

Livable attendu : <docs/scenario-final.md>

### Partie 6 : Mise en production & Présentation

#### TP 6.1 — Containerisation Docker (vision)

Objectif

Instructions

Livable attendu : <docs/deployment-vision.md>

#### TP 6.2 — Orchestration Docker Compose (conceptuelle)

Objectif

Instructions

Livable attendu : <docs/architecture-runtime.md>

#### TP 6.3 — Observabilité (dashboard)

Objectif

Instructions

Livable attendu : <docs/dashboard-grafana.png>

#### TP 6.4 — Présentation orale

Objectif

Instructions

Contenu minimal du support

## 0. Organisation générale

### 0.1. Modalités de travail

- Travail **en binômes** pendant toute la durée de l'UE.
- Le TP est **fil rouge** : toutes les parties sont liées et doivent rester cohérentes entre elles.
- Le travail demandé est **100 % analyse / modélisation / documentation** :
  - **Aucun développement applicatif n'est exigé.**
  - Il n'est pas nécessaire d'ouvrir un IDE.

- Vous ne rendrez **aucun fichier de code exécutable** (.py, .java, .ts, etc.).

## 0.2. Repository Git

Chaque binôme crée un **repository Git unique** contenant tous les livrables.

Arborescence minimale obligatoire :

```
.
├── docs/
│   ├── domaine.md
│   ├── event-storming.png
│   ├── ubiquitous-language.md
│   ├── domain-overview.md
│   ├── bounded-contexts.md
│   ├── context-map.md
│   ├── domain-concepts.md
│   ├── domain-model.md
│   ├── invariants.md
│   ├── tests-domaine.md
│   ├── architecture-hexagonale.md
│   ├── repositories.md
│   ├── api-examples.md
│   ├── integration-scenarios.md
│   ├── contracts.md
│   ├── integration-design.md
│   ├── observability.md
│   ├── scenario-final.md
│   ├── deployment-vision.md
│   ├── architecture-runtime.md
│   ├── dashboard-grafana.png
│   └── presentation-support.pdf (ou .pptx)
└── README.md
```

Tous les fichiers listés ci-dessus sont obligatoires (sauf mention contraire).

S'ils ne sont pas fournis, le TP sera considéré comme **incomplet**.

## 0.3. Règles générales

- Tous les livrables texte sont au format **Markdown** (.md) et en **français**.

- Les diagrammes peuvent être :
    - générés via un outil (Miro, Whimsical, Draw.io, etc.) puis exportés en `.png` ou `.pdf` ,
    - ou dessinés à la main puis scannés, à condition que ce soit **lisible**.
  - **Ubiquitous Language** :
    - Tous les termes métier doivent être définis dans `docs/ubiquitous-language.md` .
    - Les noms utilisés dans les diagrammes et documents doivent respecter ce glossaire.
- 

# Partie 1 : Découverte du domaine

## TP 1.1 — Choix du scénario et description du domaine

### Objectif

Choisir un domaine et poser les bases métier.

### Instructions

1. Choisissez **un seul scénario** parmi les trois :
  - Billetterie & réservation
  - E-commerce & livraison
  - Téléconsultation & prescriptions
2. Identifiez **au moins 3 rôles utilisateurs** :
  - Un rôle "direction" (ex : Responsable commercial, Directeur médical...)
  - Un rôle "opérationnel" (ex : Agent de billetterie, Préparateur de commande, Médecin...)
  - Un rôle "client final" (ex : Client, Patient...)
3. Décrivez :
  - Le **contexte métier** (secteur, enjeux, contraintes).
  - Les **problématiques principales** du domaine.

- Un **scénario métier principal** (narratif) qui servira de fil rouge.

**Livrable attendu :** docs/domaine.md

Structure **obligatoire** du fichier :

1. # Scénario choisi
  - Nom du scénario (parmi les 3 proposés).
2. # Contexte métier
  - 2 à 3 **paragraphes** décrivant le secteur, le type d'organisation, les enjeux.
3. # Rôles utilisateurs
  - Tableau avec **au minimum 3 rôles** :

| Rôle | Type (Direction / Opérationnel / Client) | Description (2–3 phrases) |
|------|------------------------------------------|---------------------------|
|      |                                          |                           |
4. # Problématiques métier
  - Liste de **3 à 5 problèmes majeurs** que le système doit adresser.
5. # Scénario fil rouge
  - Description textuelle d'un **scénario complet** (10–20 lignes) montrant une séquence "typique" du système (ex. : de la demande initiale au résultat final).

## TP 1.2 — Event Storming express

### Objectif

Identifier les événements, commandes et acteurs du flux principal.

### Instructions

1. Utilisez un board collaboratif (Miro, Mural, Whimsical...) pour :
  - Identifier **au moins 10 événements métier** (past tense, ex : BilletRéservé ).
  - Identifier **au moins 5 commandes** (imperatif, ex : RéserverBillet ).
  - Lister les **acteurs** qui initient les commandes.

2. Organisez ces éléments en un **flux métier principal** (de gauche à droite).
3. Ajoutez éventuellement quelques **branches alternatives** (erreurs, annulations...).

**Livrable attendu :** docs/event-storming.png

Contenu attendu :

- Une **capture unique** (format .png ) de votre board montrant :
    - Les événements (post-its orange, par exemple).
    - Les commandes (post-its bleus, par exemple).
    - Les acteurs (post-its jaunes, par exemple).
    - Les flèches/liaisons montrant la séquence.
  - La capture doit être **lisible** (zoom adapté, pas de texte illisible).
- 

## TP 1.3 — Glossaire (Ubiquitous Language v1)

### Objectif

Définir un langage commun pour tout le TP.

### Instructions

1. Identifiez **au moins 20 termes métier** récurrents dans votre domaine :
  - Exemples : Billet , Réservation , Client , Commande , Panier , Consultation , etc.
2. Pour chaque terme, définissez :
  - Un **nom** (en PascalCase si c'est un concept central, ex : CommandeClient ).
  - Une **définition métier précise** (2–4 phrases).
  - Un **exemple concret** ancré dans votre scénario.

**Livrable attendu :** docs/ubiquitous-language.md

Structure obligatoire :

- # Ubiquitous Language – Version 1
- Tableau avec **au moins 20 lignes** :

| Terme | Définition métier (2–4 phrases) | Exemple concret (1–2 phrases) |
|-------|---------------------------------|-------------------------------|
|       |                                 |                               |

## TP 1.4 — Identification des sous-domaines

### Objectif

Distinguer Core / Supporting / Generic domains.

### Instructions

1. À partir de vos événements, commandes, rôles et glossaire, listez vos **fonctionnalités principales** (ex : gestion des réservations, paiement, facturation, notifications...).
2. Classez chaque fonctionnalité en :
  - **Core Domain** : la source principale de valeur différenciante.
  - **Supporting Domain** : nécessaire au Core, mais non différenciant.
  - **Generic Domain** : commodity, standard, commodité.

### Livrables attendus

1. `docs/domain-overview.md`
  - Contenu obligatoire :
    - `# Vue d'ensemble du domaine`
    - `## Liste des fonctionnalités`
      - Liste bullet point de **au moins 6 fonctionnalités**.
    - `## Classification des sous-domaines`
      - Tableau :

| Fonctionnalité | Type (Core / Supporting / Generic) | Justification (2–3 phrases) |
|----------------|------------------------------------|-----------------------------|
|                |                                    |                             |

2. Schéma intégré dans le fichier (via lien ou image) ou en fichier séparé (optionnel) :
  - Diagramme simple (3 colonnes ou 3 cercles) montrant :



- Core Domain
  - Supporting Domains
  - Generic Domains
- 

## Partie 2 : Bounded Contexts & Context Map

### TP 2.1 — Identification des Bounded Contexts

#### Objectif

Structurer le système en contextes cohérents.

#### Instructions

1. Regroupez vos événements, commandes et fonctionnalités en **Bounded Contexts** cohérents.
2. Donnez des **noms explicites** à chaque BC (ex : `ContexteRéservation` , `ContextePaiement` , etc.).
3. Classifiez chaque BC en Core / Supporting / Generic (lié à la Partie 1.4).

**Livrable attendu :** `docs/bounded-contexts.md`

Structure obligatoire :

- `# Bounded Contexts`
- Tableau avec **au moins 3 Bounded Contexts** :

| Bounded Context | Type (Core / Supporting / Generic) | Rôle / Responsabilité principale (3–5 phrases) |
|-----------------|------------------------------------|------------------------------------------------|
|                 |                                    |                                                |

### TP 2.2 — Choix des patterns de relation & Context Map

#### Objectif

Définir les relations entre contextes et leurs mécanismes d'intégration.

## Instructions

1. Identifiez les **relations** entre vos BCs (qui dépend de qui, qui fournit quoi à qui).
2. Pour chaque relation, choisissez un **pattern** :
  - Partnership, Conformist, Anticorruption Layer, Customer/Supplier, etc.
3. Décrivez **au moins 3 intégrations techniques envisagées** :
  - Exemples : API REST, événements via broker, files, etc.

## Livrable attendu : `docs/context-map.md`

Contenu obligatoire :

1. `# Context Map`
2. `## Schéma général`
  - Diagramme context map (inséré sous forme d'image ou ASCII) montrant les BCs et leurs liens.
3. `## Relations et patterns`
  - Tableau :

| Contexte source | Contexte cible | Pattern de relation | Justification (3–5 phrases) |
|-----------------|----------------|---------------------|-----------------------------|
|                 |                |                     |                             |
4. `## Intégrations techniques envisagées`
  - Liste d'**au moins 3 intégrations** :
    - Pour chacune : type (REST, events...), BC impliqués, cas d'usage.

## TP 2.3 — Ubiquitous Language v2

### Objectif

Affiner le langage pour prendre en compte les Bounded Contexts.

## Instructions

1. Réutilisez `docs/ubiquitous-language.md` et **mettez-le à jour** :
  - Ajouter / modifier des termes si nécessaire.

- Associer chaque terme à un **contexte principal** lorsqu'il est contextuel.
2. Ajouter une section « Termes par contexte ».

**Livrable attendu : mise à jour de** `docs/ubiquitous-language.md`

Contenu supplémentaire obligatoire :

- Nouvelle section :

```
## Termes par contexte
```

| Terme | Contexte principal | Commentaire (facultatif) |
|-------|--------------------|--------------------------|
| ----- | -----              | -----                    |
| --    |                    |                          |
|       |                    |                          |

## TP 2.4 — Alignement conceptuel du modèle

### Objectif

Clarifier quelques éléments métier clés sans les coder.

### Instructions

1. Choisissez **2 entités métier importantes** et **1 objet valeur** dans votre domaine.
2. Pour chaque entité :
  - Décrire ses **attributs principaux** (nom + type métier + description).
  - Décrire **au moins 1 invariant métier** (règle qui doit toujours être vraie).
3. Pour l'objet valeur :
  - Expliquer pourquoi il est **immuable**.
  - Décrire ses propriétés.

**Livrable attendu :** `docs/domain-concepts.md`

Structure obligatoire :

- `# Entités et Objets Valeur – Vue conceptuelle`
- `## Entités`

- Pour chaque entité :

- Nom de l'entité
- Tableau d'attributs :

| Attribut | Type métier (pas technique) | Description (1–2 phrases) |
|----------|-----------------------------|---------------------------|
|          |                             |                           |

- Liste d'**au moins 1 invariant** par entité.
- **## Objet Valeur**
  - Nom de l'OV
  - Propriétés
  - Explication de l'immutabilité (2–3 phrases)

## Partie 3 : Modélisation tactique

### TP 3.1 — Entités & Objets Valeur (modèle)

#### Objectif

Formaliser le modèle conceptuel du domaine.

#### Instructions

1. Définir **3 entités** avec identifiant unique métier (ex : `RéservationId` , `ClientId` ).
2. Définir **3 objets valeur** immuables.
3. Représenter les relations sous forme de **diagramme UML de classes** (conceptuel).

**Livrable attendu :** `docs/domain-model.md`

Contenu obligatoire :

1. **# Modèle de domaine**
2. **## Entités**
  - Tableau :

| Entité | Description métier (3–5 phrases) | Identifiant métier |
|--------|----------------------------------|--------------------|
|        |                                  |                    |

### 3. ## Objets Valeur

- Tableau :

| Objet Valeur | Description métier (2–4 phrases) | Propriétés principales |
|--------------|----------------------------------|------------------------|
|              |                                  |                        |

### 4. ## Diagramme UML (conceptuel)

- Image intégrée (ou lien vers fichier) représentant :
  - Les 3 entités
  - Les 3 objets valeur
  - Les relations (association, composition, etc.)

## TP 3.2 — Agrégats & Invariants

### Objectif

Identifier les agrégats clés et leurs règles.

### Instructions

1. Sélectionner **2 agrégats clés** dans votre domaine.
2. Pour chaque agrégat, définir :
  - L'**entité racine**.
  - Les **frontières** (quelles entités/OV font partie de l'agrégat).
  - **Au moins 3 invariants** métier.

**Livrable attendu :** [docs/invariants.md](#)

Contenu obligatoire :

- [# Agrégats et invariants](#)
- Pour chaque agrégat :
  - Nom de l'agrégat
  - Racine de l'agrégat

- Liste des entités/OV internes

- Tableau des invariants :

| Invariant | Description métier (3–4 phrases) | Conséquence si non respecté |
|-----------|----------------------------------|-----------------------------|
|           |                                  |                             |

- **## Schéma UML des agrégats**
  - Un diagramme (image) montrant les agrégats, leurs frontières et racines.

## TP 3.3 — Tests de domaine (Given/When/Then textuels)

### Objectif

Valider les invariants sous forme de scénarios BDD.

### Instructions

1. Pour **chaque invariant** défini dans **invariants.md**, écrire :
  - 1 scénario **happy path** (invariant respecté).
  - 1 scénario **sad path** (invariant violé).
2. Utiliser un format Given/When/Then.

**Livrable attendu :** **docs/tests-domaine.md**

Contenu obligatoire :

- **# Scénarios de test de domaine**
- Pour chaque invariant (numéroté) :
  - **### Invariant X : <Nom>**
  - **#### Scénario 1 – Happy path**
    - Given ...
    - When ...
    - Then ...

- ##### Scénario 2 – Sad path
    - Given ...
    - When ...
    - Then ...
- 

## Partie 4 : Architecture hexagonale (conceptuelle)

### TP 4.1 — Squelette Hexagonal

#### Objectif

Décrire l'architecture hexagonale du système.

#### Instructions

1. Définir les responsabilités de chaque couche :
  - **Domain**
  - **Application**
  - **Adapters** (REST, persistance, messaging, etc.)
2. Illustrer comment une **commande métier** traverse ces couches.

**Livrable attendu :** `docs/architecture-hexagonale.md`

Contenu obligatoire :

- `# Architecture hexagonale`
  - `## Description des couches`
    - 1 paragraphe par couche (Domain, Application, Adapters).
  - `## Exemple de flux (commande → réponse)`
    - Narration d'un cas concret (5–10 lignes).
    - Schéma (optionnel mais fortement recommandé).
- 

### TP 4.2 — Repositories (conceptuels)

## Objectif

Formaliser la persistance du domaine.

## Instructions

1. Décrire **2 repositories** métier (ex : `RéservationRepository` , `ClientRepository` ).
2. Pour chaque repository :
  - Décrire les opérations principales (Create, Read, Update, Delete).
  - Préciser les **règles métier** associées aux opérations (pas de SQL, pas de code).

**Livrable attendu :** `docs/repositories.md`

Contenu obligatoire :

- `# Repositories métier`
- Pour chaque repository :
  - Tableau :

| Operation métier | Description (3–4 phrases) | Contraintes / règles métier |
|------------------|---------------------------|-----------------------------|
|                  |                           |                             |

## TP 4.3 — Exposition REST (maquettes uniquement)

### Objectif

Maquetter une API REST sans l'implémenter.

### Instructions

1. Choisir **2 endpoints REST** :
  - 1 pour la création d'un agrégat.
  - 1 pour la lecture d'un état.
2. Décrire :
  - L'URL, la méthode HTTP.
  - Un exemple de **requête** (JSON).
  - Un exemple de **réponse** (JSON).



3. Respecter l'**Ubiquitous Language** pour les noms.

**Livrable attendu :** `docs/api-examples.md`

Contenu obligatoire :

- `# Exemples d'API REST`
  - Pour chaque endpoint :
    - `## Endpoint X`
      - Méthode + URL
      - Description
      - Exemple de requête (bloc JSON)
      - Exemple de réponse (bloc JSON)
- 

## TP 4.4 — Tests d'intégration (scénarios sur papier)

### Objectif

Décrire un scénario end-to-end à travers plusieurs couches et contexts.

### Instructions

1. Définir au moins **1 scénario end-to-end** :
  - De la réception d'une requête externe jusqu'à la mise à jour du domaine.
2. Représenter ce scénario sous forme :
  - de narration,
  - et/ou de **diagramme de séquence UML**.

**Livrable attendu :** `docs/integration-scenarios.md`

Contenu obligatoire :

- `# Scénarios d'intégration`
  - Texte narratif du scénario (10–20 lignes).
  - Schéma de séquence (recommandé, image intégrée).
-

# Partie 5 : Intégration entre contexts & qualité

## TP 5.1 — Définition des contrats

### Objectif

Formaliser les échanges entre deux contexts.

### Instructions

1. Choisir **2 Bounded Contexts**.
2. Définir un contrat d'échange entre eux :
  - Soit en **OpenAPI (YAML)**,
  - Soit sous forme de **schéma JSON** (request/response),
  - Soit sous forme de **schéma d'événement** (JSON/Avro simplifié).

**Livrable attendu :** `docs/contracts.md`

Contenu obligatoire :

- `# Contrats d'échange entre contexts`
- Pour chaque contrat :
  - Context source / cible
  - Type d'échange (REST, event...)
  - Schéma du message (en YAML ou JSON, formaté dans un bloc de code)

## TP 5.2 — Implémentation de l'intégration (design)

### Objectif

Concevoir comment l'intégration fonctionnerait, sans la développer.

### Instructions

1. Décrire :
  - 1 intégration **REST** (avec séquence d'appel).

- 1 intégration **par événements** (publish/subscribe).
2. Illustrer par :
- Un **diagramme de flux**,
  - Et une **description narrative** (5–10 lignes par intégration).

**Livrable attendu :** `docs/integration-design.md`

Contenu obligatoire :

- `# Design des intégrations inter-contextes`
- `## Intégration REST`
  - Schéma (image)
  - Narration
- `## Intégration par événements`
  - Schéma (image)
  - Narration

## TP 5.3 — Observabilité (conceptuelle)

### Objectif

Prévoir comment observabilité et monitoring seraient intégrés.

### Instructions

1. Définir ce qu'est un **Correlation ID** dans votre système :
  - Où il est créé, où il est propagé.
2. Proposer **3 métriques métier** exposables sur `/metrics`.
3. Proposer un format de **logging structuré** (ex : JSON log).

**Livrable attendu :** `docs/observability.md`

Contenu obligatoire :

- `# Observabilité`
- `## Correlation ID`
  - Description (3–5 phrases)

- **## Métriques métier**

- Tableau :

| Nom de la métrique | Description | Type (counter / gauge / histogram) |
|--------------------|-------------|------------------------------------|
|                    |             |                                    |

- **## Logging structuré**

- Exemple de log JSON (bloc de code).

## TP 5.4 — Validation par scénario

### Objectif

Valider un scénario complet inter-contextes.

### Instructions

1. Choisir un **scénario complet** (ex : Commande → Paiement → Livraison).
2. Décrire :
  - Les contextes impliqués,
  - Les étapes majeures,
  - Les événements clés.

**Livrable attendu :** **docs/scenario-final.md**

Contenu obligatoire :

- **# Scénario complet inter-contextes**
- Description narrative détaillée (15–30 lignes).
- Liste des événements déclenchés (dans l'ordre).
- Rappel des invariants concernés.

## Partie 6 : Mise en production & Présentation

### TP 6.1 — Containerisation Docker (vision)

## Objectif

Décrire comment on déploierait le système avec Docker.

## Instructions

1. Imaginer un **service par Bounded Context**.
2. Expliquer :
  - Quels services seraient conteneurisés.
  - Comment ils seraient configurés (à haut niveau).

**Livrable attendu :** `docs/deployment-vision.md`

Contenu obligatoire :

- `# Vision de déploiement`
- Description écrite (10–20 lignes).
- Exemple **annoté** de Dockerfile (pseudo-code dans un bloc de code, non fonctionnel).

## TP 6.2 — Orchestration Docker Compose (conceptuelle)

### Objectif

Montrer une vue runtime des services.

### Instructions

1. Imaginer un `docker-compose.yml` avec :
  - Au moins **3 services** : API, DB, broker.
2. Décrire les liens entre services.

**Livrable attendu :** `docs/architecture-runtime.md`

Contenu obligatoire :

- `# Architecture runtime`
- Description textuelle des services.
- Schéma (image) montrant les services et leurs liens.

## TP 6.3 — Observabilité (dashboard)

### Objectif

Concevoir un dashboard de monitoring.

### Instructions

1. Imaginer un dashboard Grafana :
  - 1 graphique **technique** (ex : temps de réponse, erreurs...)
  - 1 graphique **métier** (ex : commandes validées par heure)
2. Le représenter sous forme de mockup.

**Livrable attendu :** `docs/dashboard-grafana.png`

Contenu obligatoire :

- Image (mockup ou capture d'un dashboard fictif)
- 2 graphiques **clairement annotés** :
  - Nom de la métrique
  - Axe X/Y

## TP 6.4 — Présentation orale

### Objectif

Présenter l'ensemble de votre travail.

### Instructions

- Durée : **10 minutes de présentation** + 5 minutes de questions.
- Support obligatoire : `docs/presentation-support.pdf` (ou `.pptx`).

### Contenu minimal du support

- Slide 1 : Contexte & scénario choisi.
- Slide 2–3 : Context map & Bounded Contexts.
- Slide 4 : Un agrégat clé + invariants.
- Slide 5 : Scénario inter-contextes.

- Slide 6 : Vision d'architecture (hexagonale + runtime).
- Slide 7 : Observabilité & métriques.